

HUS.HAD

Hanoi University of Science

Mục lục

0 Contest 1
- 0.1 Template 1
- 0.2 Shell: compile/run 2
- 0.3 Fast IO 2
- 0.4 stress.sh 2
- 0.5 Gen helper 2
- 0.6 Checker skeleton 2
- 0.7 Troubleshoot + edge cases 2

1 Flow & Matching 2
- 1.1 Flow 2
  - 1.1.1 Dinic with scaling + min cut 2
  - 1.1.2 MCMF 2
  - 1.1.3 Min-cost circulation (cost scaling) 2
  - 1.1.4 Flow models 3
  - 1.1.5 Global min cut (Stoer–Wagner) 3
  - 1.1.6 Gomory–Hu tree 3
- 1.2 Matching 3
  - 1.2.1 Hopcroft–Karp 3
  - 1.2.2 Hungarian 3
  - 1.2.3 General matching (blossom) 3

2 Graph 3
- 2.1 Connectivity 3
  - 2.1.1 2-SAT 3
  - 2.1.2 Lowlink: bridges, articulation points, BCC 3
  - 2.1.3 Euler path / circuit 3
- 2.2 Paths & spanning trees 3
  - 2.2.1 SPFA 3
  - 2.2.2 Directed MST 4
  - 2.2.3 Manhattan MST 4
  - 2.2.4 Matrix-tree theorem 4
- 2.3 Trees 4
  - 2.3.1 HLD 4
  - 2.3.2 Centroid decomposition 4
  - 2.3.3 Virtual tree 4
- 2.4 Hard 4
  - 2.4.1 Dominator tree 4
  - 2.4.2 Max clique 4

3 Data Structures 4
- 3.1 Segment trees 4
  - 3.1.1 Lazy segment tree 4
  - 3.1.2 Persistent segment tree 4
  - 3.1.3 Li Chao tree 5
- 3.2 Balanced trees 5
  - 3.2.1 Implicit treap 5
  - 3.2.2 Link-cut tree 5
- 3.3 Other 5
  - 3.3.1 Rollback DSU 5
  - 3.3.2 pb_ds 5
  - 3.3.3 Mo's algorithm 5

4 Strings 5
- 4.1 Z-function 5
- 4.2 Rolling hash 5
- 4.3 Suffix array + LCP 5
- 4.4 Aho–Corasick 5
- 4.5 Suffix automaton 6
- 4.6 Eertree 6
- 4.7 Lyndon factorisation (Duval) 6

5 Geometry 6
- 5.1 Point / line primitives 6
- 5.2 Polygon 6
- 5.3 Convex hull 6
- 5.4 Circles 6
- 5.5 Half-plane intersection 6
- 5.6 Closest pair / kd-tree 6
- 5.7 3D 6

6 Math 6
- 6.1 Number theory 6
  - 6.1.1 Modint 6
  - 6.1.2 Bigint 6
  - 6.1.3 Ext-gcd, CRT 7
  - 6.1.4 Sieve (factorable) 7
  - 6.1.5 Miller–Rabin 7
  - 6.1.6 Pollard rho 7
  - 6.1.7 Discrete log / discrete sqrt 8
  - 6.1.8 Floor sum 8
- 6.2 Linear algebra 8
  - 6.2.1 Gauss–Jordan 8
  - 6.2.2 Simplex 8
- 6.3 Polynomials & transforms 8
  - 6.3.1 FFT 8
  - 6.3.2 NTT 8
  - 6.3.3 FWHT 9
  - 6.3.4 FPS: inv, log, exp, div 9
  - 6.3.5 Berlekamp–Massey + Bostan–Mori 9
  - 6.3.6 Lagrange interpolation 9
- 6.4 Numerical 9
  - 6.4.1 Simpson integration 9

7 Interactive & Multi-pass 9
- 7.1 Checklists 9
  - 7.1.1 Interactive 9
  - 7.1.2 Multi-pass 9
  - 7.1.3 Noise taxonomy 9
- 7.2 Codes 9
  - 7.2.1 Linear codes: Reed–Solomon, GF(2), stuck bits 9
  - 7.2.2 Bit-level codes: self-delimiting, VT, pairs, checksum 9
  - 7.2.3 Hamming SECDED 9
- 7.3 Encoding 9
  - 7.3.1 Rank / unrank 9
  - 7.3.2 Tree canonical form (AHU) 9

8 Misc 9
- 8.1 DP optimisation: D&C, Knuth, Aliens 9
- 8.2 Ternary search + simulated annealing 9

9 Math Reference 9
- 9.1 Counting 9
- 9.2 Number theory 9
- 9.3 Sums & generating functions 9
- 9.4 Games 9
- 9.5 Probability & identities 9
- 9.6 Techniques prompt list 9

0 Contest

0.1 Template

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;
/*
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
  tree_order_statistics_node_update> ordered_set;
*/
typedef long long ll;
typedef long double ld;
template<class T> using PQMax = priority_queue<T>;
template<class T> using PQMin = priority_queue<T,
  vector<T>, greater<T>>;
template<class T1, class T2>
void maximize(T1 &a, T2 b){ if (b > a) a = b; }
template<class T1, class T2>
void minimize(T1 &a, T2 b){ if (b < a) a = b; }
template<class T> void read(T &number) {
  bool negative = false; number = 0;
  static int c = getchar();
  while (c != '-' && !isalnum(c)) c = getchar();
  if (c == '-') negative = true, c = getchar();
  for (; (c >= '0' && c <= '9'); c = getchar())
    number = number * 10 + c - '0';
  if (negative) number = -number;
}
template<class T, class ...Ts>
void read(T &a, Ts&... args){
  read(a); read(args...);
}

#define fi first
#define se second
#define FOR(type,i,a,b) for(type i=(a); i<=(b); i++)
#define REV(type,i,b,a) for(type i=(b); i>=(a); i--)
#define EFOR(type,i,a,b) for(type i=(a); i<(b); i++)
#define EREV(type,i,b,a) for(type i=(b); i>(a); i--)

#define testBit(n, bit) (((n) >> (bit)) & 1)
#define flipBit(n, bit) ((n) ^ (1ll << (bit)))
#define cntBit(n) __builtin_popcountll(n)
#define log2(n) (63 - __builtin_clzll(n))

#define CURRENT_TIMESTAMP
  chrono::steady_clock::now().time_since_epoch().count()
#define randomize mt19937_64 mt(CURRENT_TIMESTAMP)

/*
 * remember to fill in:
 *   #define MAX ???
 *   #define MOD ???
 *
```

```
 *   int main() {
 *     ios_base::sync_with_stdio(0); cin.tie(nullptr);
 *   }
 */
```

0.2 Shell: compile/run

```
# compile
g++ $1.cpp --std=c++17 -Wall -Wextra -O2 -o $1
diff output.txt answer.txt
# before running shell file
chmod 700 any_shell_file.sh
```

0.3 Fast IO

0.4 stress.sh

0.5 Gen helper

0.6 Checker skeleton

0.7 Troubleshoot + edge cases

1 Flow & Matching

1.1 Flow

1.1.1 Dinic with scaling + min cut

```
struct Flow{
  struct Edge{int dest, rev; ll capa;};
  const ll oo = 1ll << 60;
  vector<vector<Edge>> adj;
  vector<pair<int,int>> where;
  int n, s, t;
  vector<int> level, ptr;
  void init(int n, int s, int t){ this->n=n; this->s=s; this->t=t; adj.assign(n,{}); where.clear(); level.resize(n); ptr.resize(n); }
  void addEdge(int u, int v, ll c){
    where.push_back({u, (int)adj[u].size()});
    adj[u].push_back({v, (int)adj[v].size(), c});
    adj[v].push_back({u, (int)adj[u].size() - 1, 0});
  }
  bool bfs(ll lim){
    fill(level.begin(), level.end(), -1);
    queue<int> q; q.push(t); level[t] = 0;
    while (!q.empty() && level[s] < 0){
      int v = q.front(); q.pop();
      for (auto &e: adj[v])
        if (level[e.dest] < 0 && adj[e.dest][e.rev].capa >= lim)
          level[e.dest] = level[v] + 1, q.push(e.dest);
    }
    return level[s] >= 0;
  }
  ll dfs(int v, ll f, ll lim){
    if (v == t) return f;
    ll pushed = 0;
    for (int &i = ptr[v]; i < (int)adj[v].size(); i++){
      Edge &e = adj[v][i];
      if (e.capa < lim || level[e.dest] != level[v] - 1) continue;
      ll got = dfs(e.dest, min(f - pushed, e.capa) / lim * lim, lim);
      if (got){ e.capa -= got; adj[e.dest][e.rev].capa += got; pushed += got; if (pushed == f) return pushed; }
    }
    return pushed;
  }
  ll calculate(){
    ll ans = 0;
    for (ll lim = oo; lim; lim >>= 1)
      while (bfs(lim)){ fill(ptr.begin(), ptr.end(), 0); ans += dfs(s, oo, lim); }
    return ans;
  }
  ll capOf(int id){ return adj[where[id].first][where[id].second].capa; }
};
```

1.1.2 MCMF

Uses SPFA (§2.2.1). `calculate(F)`: cheapest flow of at most F; no limit = min-cost max-flow. Each augmentation is optimal for its flow value. No negative cycles; `MCMF<__int128>` if the total cost can overflow.

```
// successive shortest paths (SPFA on the residual graph); no negative cycles. C = type of the total cost
template<class C = ll> struct MCMF{
  struct Edge{int from, dest, oppo; ll capa, cost;};
  vector<Edge> edges, resi;
  vector<vector<int>> graph;
  SPFA sp;
  int n, s, t;

  void init(int n, int s, int t){
    this -> n = n; this -> s = s; this -> t = t;
    edges.clear(); graph.assign(n, {});
  }

  int addEdge(int u, int v, ll capa, ll cost){ // returns edge id
    int m = edges.size();
    edges.push_back({u, v, m + 1, capa, cost});
    edges.push_back({v, u, m, 0, -cost});
    graph[u].push_back(m); graph[v].push_back(m + 1);
    return m / 2;
  }

  pair<ll, C> calculate(ll limit = 1ll << 62){ // {flow, cost}, flow <= limit
    resi = edges; ll flow = 0; C cost = 0;
    auto residual = [&](int u, auto relax){
      for (int idx: graph[u]) if (resi[idx].capa > 0) relax(resi[idx].dest, resi[idx].cost, idx);
    };
    while (flow < limit){
      sp.run(n, s, residual);
      if (sp.dist[t] >= SPFA::oo) break;
      ll push = limit - flow;
      for (int v = t; v != s; v = resi[sp.par[v]].from) push = min(push, resi[sp.par[v]].capa);
      for (int v = t; v != s; v = resi[sp.par[v]].from){
        int e = sp.par[v];
        resi[e].capa -= push; resi[resi[e].oppo].capa += push;
      }
      flow += push; cost += (C)push * sp.dist[t];
    }
    return {flow, cost};
  }

  ll flowOn(int id){ return resi[2 * id + 1].capa; }
};
```

1.1.3 Min-cost circulation (cost scaling)

Bounds $l \le f \le r$, any costs, no source/sink. $s \to t$ flow of exactly F: add $t \to s$ with $[F, F]$, cost 0. Supplies b_v: extra vertex X, edges $X \to v$ $[b_v, b_v]$ ($b_v > 0$), $v \to X$ $[-b_v, -b_v]$ ($b_v < 0$). Infeasible $\Rightarrow$ `solve().first = false`.

```
// Min-cost circulation with bounds l <= f <= r and any costs; cost-scaling push-relabel (koosaga).
// O(V^2 E log(VC)), independent of capacities. After solve(): flowOf(i), p = duals.
struct MinCostCirculation {
  struct In { int s, t; ll l, r, c; };
  struct Edge { int to, rev; ll rem, c; };
  int n; vector<In> in; vector<vector<Edge>> g;
  vector<int> pos; vector<ll> p;
  MinCostCirculation(int n): n(n), g(n), p(n) {}
  int addEdge(int s, int t, ll l, ll r, ll c) {
    in.push_back({s, t, l, r, c}); return in.size() - 1;
  }
  ll flowOf(int i) {
    auto &e = in[i];
    if (e.s == e.t) return e.c < 0 ? e.r : e.l;
    // self-loops never enter the graph
    return e.r - g[e.s][pos[i]].rem;
  }

  pair<bool, __int128> solve() {
    // 1. feasible circulation: lower-bound reduction + max flow
    Flow f; f.init(n + 2, n, n + 1);
    vector<ll> ex(n); ll need = 0;
```

```
    for (auto &e: in) if (e.s != e.t) f.addEdge(e.s, e.t,
    ↪  e.r - e.l), ex[e.t] += e.l, ex[e.s] -= e.l;
    for (int v = 0; v < n; v++)
      if (ex[v] > 0) f.addEdge(n, v, ex[v]), need +=
      ↪  ex[v];
      else if (ex[v] < 0) f.addEdge(v, n + 1, -ex[v]);
    if (f.calculate() != need) return {false, 0};
    // 2. residual graph, costs scaled by n+1 so that eps
    ↪  <= 1 means optimal
    pos.assign(in.size(), -1);
    for (int i = 0, k = 0; i < (int)in.size(); i++) {
      auto &e = in[i]; if (e.s == e.t) continue;
      ll cur = e.r - f.capOf(k++);                    //
      ↪  flow on e, in [l, r]
      pos[i] = g[e.s].size();
      g[e.s].push_back({e.t, (int)g[e.t].size(), e.r -
      ↪  cur, e.c * (n + 1)});
      g[e.t].push_back({e.s, pos[i], cur - e.l, -e.c * (n
      ↪  + 1)});
    }
    // 3. refine: eps /= 8 per phase; saturate negative
    ↪  edges, then push-relabel the excess away
    auto rc = [&](Edge &e, int u) { return e.c + p[u] -
    ↪  p[e.to]; };
    for (ll eps; ; ) {
      eps = 0;
      for (int u = 0; u < n; u++) for (auto &e: g[u]) if
      ↪  (e.rem > 0) eps = max(eps, -rc(e, u));
      if (eps <= 1) break;
      eps = max(1ll, eps >> 3);
      vector<ll> excess(n); vector<int> ptr(n); queue<int>
      ↪  q;
      auto push = [&](Edge &e, int u, ll x) {
        e.rem -= x; g[e.to][e.rev].rem += x; excess[u] -=
        ↪  x; excess[e.to] += x;
        if (excess[e.to] > 0 && excess[e.to] <= x)
        ↪  q.push(e.to);   // just became active
      };
      for (int u = 0; u < n; u++) for (auto &e: g[u]) if
      ↪  (e.rem > 0 && rc(e, u) < 0) push(e, u, e.rem);
      while (!q.empty()) {
        int u = q.front(); q.pop();
        while (excess[u] > 0) {
          for (; ptr[u] < (int)g[u].size() && excess[u] >
          ↪  0; ptr[u]++) {
            Edge &e = g[u][ptr[u]];
            if (e.rem > 0 && rc(e, u) < 0) push(e, u,
            ↪  min(e.rem, excess[u]));
          }
          if (excess[u] > 0) {                        //
          ↪  relabel: cheapest residual edge gets rc =
          ↪  -eps
            ptr[u] = 0; p[u] = LLONG_MIN / 4;
            for (auto &e: g[u]) if (e.rem > 0) p[u] =
            ↪  max(p[u], p[e.to] - e.c - eps);
          }
        }
      }
    }
    // 4. duals (Bellman-Ford on the residual graph, real
    ↪  costs) and the answer
    for (auto &v: g) for (auto &e: v) e.c /= n + 1;
    fill(p.begin(), p.end(), 0);
    for (bool upd = true; upd; ) { upd = false;
      for (int u = 0; u < n; u++) for (auto &e: g[u])
        if (e.rem > 0 && p[u] + e.c < p[e.to]) p[e.to] =
        ↪  p[u] + e.c, upd = true; }
    __int128 ans = 0;
    for (int i = 0; i < (int)in.size(); i++) ans +=
    ↪  (__int128)in[i].c * flowOf(i);
    return {true, ans};
  }
};
```

1.1.4 Flow models

1.1.5 Global min cut (Stoer–Wagner)

1.1.6 Gomory–Hu tree

1.2 Matching

1.2.1 Hopcroft–Karp

1.2.2 Hungarian

1.2.3 General matching (blossom)

2 Graph

2.1 Connectivity

2.1.1 2-SAT

```
class TWOSET{
private:
  inline static constexpr int oo = 1e9;
  int n, all, m, cntGr = 0, time = 0;
  vector<vector<int>> graph; vector<int> grIdx, low, num;
  vector<bool> answer; stack<int> st;

  inline int getIdx(int u){return (u > 0) ? u : (-u +
  ↪  n);};
  void initTarjan()
    { low = num = vector<int>(all, false); time=cntGr=0; }
  void tarjan(int node){
    low[node] = num[node] = ++time; st.push(node);
    for (int child: graph[node])
      if (num[child]) minimize(low[node], num[child]);
      else{tarjan(child); minimize(low[node],low[child]);}
    if (num[node] == low[node]){
      ++cntGr; int v = -1;
      while (v != node){
        v=st.top();st.pop();
        grIdx[v]=cntGr;low[v]=num[v]=oo;
      } } }
public:
  TWOSET(){}
  void init(int n){
    this -> n = n; m = 0; all = n * 2 + 1;
    graph = vector<vector<int>>(all, vector<int>());
    grIdx = vector<int>(all, 0);
  }
  void addEdge(int u, int v){
    graph[getIdx(-u)].push_back(getIdx(v));
    graph[getIdx(-v)].push_back(getIdx(u));
  }
  bool checkSatisfiable(){
    initTarjan();
    for (int node = 1; node < all; node++)
      if (low[node] == 0) tarjan(node);
    answer = vector<bool>(n + 1);
    for (int i = 1; i <= n; i++){
      int u = getIdx(i), v = getIdx(-i);
      if (grIdx[u] == grIdx[v]) return false;
      answer[i] = grIdx[u] < grIdx[v];
    }
    return true;
  }
  bool getAnswer(int u){ return answer[u]; }
};
```

2.1.2 Lowlink: bridges, articulation points, BCC

2.1.3 Euler path / circuit

2.2 Paths & spanning trees

2.2.1 SPFA

Path: walk `par` back from t (edge ids, so keep each edge's tail). Other `adj`: filter edges (`if (w <= lim)`), change weights (min mean cycle: $w - \lambda$), implicit graphs (grids).

```
// SPFA with SLF/LLL deque heuristics. The caller supplies
↪  the edges:
// adj(u, relax) must call relax(v, w, id) for every usable
↪  edge u -> v.
// Returns true if a negative cycle is reachable from s;
↪  with detect, dist = -oo on/after it.
struct SPFA {
  static constexpr ll oo = LLONG_MAX / 4;
  vector<ll> dist; vector<int> par;              // par[u]
  ↪  = id of the edge into v, -1 if none

  template<class Adj> bool run(int n, int s, Adj adj, bool
  ↪  detect = false) {
    dist.assign(n, oo); par.assign(n, -1);
    vector<int> len(n), cnt(n); vector<char> inq(n);
    ↪  vector<int> bad;
    const int LIM = sqrt(n);
    deque<int> q{s}; dist[s] = 0; inq[s] = 1;
```

```
    __int128 qSum = 0;                          // sum of dist over queued vertices
    while (!q.empty()) {
      while (dist[q.front()] > dist[q.back()])
        q.push_back(q.front()), q.pop_front();
      int u = q.front(); q.pop_front(); inq[u] = 0; qSum -= dist[u];
      adj(u, [&](int v, ll w, int id) {
        ll nd = dist[u] + w;
        if (nd >= dist[v]) return;
        qSum += inq[v] ? nd - dist[v] : nd;
        dist[v] = nd; par[v] = id; len[v] = len[u] + 1;
        if (len[v] >= n) { bad.push_back(v); if (!inq[v]) qSum -= nd; return; }   // >= n edges: negative cycle
        if (inq[v]) return;
        inq[v] = 1;
        if (++cnt[v] > LIM || (__int128)nd * (ll)q.size() > qSum) q.push_back(v);
        else if (q.empty() || nd <= dist[q.front()]) q.push_front(v);
        else q.push_back(v);
      });
      if (!bad.empty() && !detect) return true;
    }
    if (bad.empty()) return false;
    for (int v: bad) dist[v] = -oo, q.push_back(v);
    while (!q.empty()) {
      int u = q.front(); q.pop_front();
      adj(u, [&](int v, ll, int) { if (dist[v] != -oo) dist[v] = -oo, q.push_back(v); });
    }
    return true;
  }
};

inline void _example() {
  int n = 4, s = 0;
  vector<vector<array<ll, 3>>> g(n);            // g[u] = {v, w, id}; shuffle each g[u] once
  SPFA sp;
  sp.run(n, s, [&](int u, auto relax) { for (auto [v, w, id]: g[u]) relax(v, w, id); });
}
```

2.2.2 Directed MST

2.2.3 Manhattan MST

2.2.4 Matrix-tree theorem

2.3 Trees

2.3.1 HLD

2.3.2 Centroid decomposition

2.3.3 Virtual tree

2.4 Hard

2.4.1 Dominator tree

2.4.2 Max clique

3 Data Structures

3.1 Segment trees

3.1.1 Lazy segment tree

3.1.2 Persistent segment tree

```
namespace Persistent{
template<class T> struct Node {
  Node *l, *r; T val;
  Node(): Node(T()) {}
  Node(T val): Node(val, nullptr, nullptr) {}
  Node(T val, Node* l, Node* r): l(l), r(r), val(val) {}
};
struct Range{
  inline static int N; int l, r, mid;
  inline static void init(int N) {Range::N = N;}
  Range(): Range(N) {} Range(int n): Range(1, n){}
  Range(int l, int r): l(l), r(r) {mid = (l+r)/2;}
  inline bool isSingle() {return l == r;}
  inline bool contains(int x) {return l<=x&&x<=r;}
  inline bool isInside(int L, int R) {return L<=l&&r<=R;}
  inline bool noRelate(int L, int R) {return r<L || R<l;}
  inline Range L() {return {l, mid};}
  inline Range R() {return {mid+1, r};}
};
template<class T> struct SegTree {
  static const T DEF_VAL;
  inline static T process(T a, T b);
  vector<Node<T>*> trees;
  SegTree() {}
  template<class T2>
  Node<T>* _buildTree(T2 *arr, Range rng) {
    if (rng.isSingle())
      return new Node<T>(arr[rng.l]);
    Node<T>* l = _buildTree(arr, rng.L());
    Node<T>* r = _buildTree(arr, rng.R());
    return new Node<T>(process(l -> val, r -> val), l, r);
  }
  Node<T>* _newRoot(Node<T>* root) {
    return new Node<T>(*root);
  }
  Node<T>* _update(Range rng, Node<T>* node, int pos, T val) {
    if (rng.isSingle())
      return new Node<T>(val);
    Node<T> *l = node -> l, *r = node -> r;
    if (rng.L().contains(pos))
      l = _update(rng.L(), l, pos, val);
    else
      r = _update(rng.R(), r, pos, val);
    return new Node<T>(process(l -> val, r -> val), l, r);
  }
  T _get(Range rng, Node<T>* node, int L, int R) {
    if (rng.noRelate(L, R)) return DEF_VAL;
    if (rng.isInside(L, R)) return node -> val;
    return process(
      _get(rng.L(), node -> l, L, R),
      _get(rng.R(), node -> r, L, R)
    );
  }

  template<class T2>void buildTree(T2* arr) {
    trees = {_buildTree(arr, Range())};
  }
  void addTree(int from) {
    assert(from < (int)trees.size());
    trees.push_back(_newRoot(trees[from]));
  }
  void update(int from, int pos, T val) {
    assert(from < (int)trees.size());
    trees[from] = _update(Range(), trees[from], pos, val);
  }
  T get(int from, int L, int R) {
    assert(from < (int)trees.size());
    return _get(Range(), trees[from], L, R);
  }
};
}
// remember to declare:
// template<> const T Persistent::SegTree<T>::DEF_VAL
// template<> T Persistent::SegTree<T>::process(T a, T b)
```

3.1.3 Li Chao tree

3.2 Balanced trees

3.2.1 Implicit treap

3.2.2 Link-cut tree

3.3 Other

3.3.1 Rollback DSU

3.3.2 pb_ds

3.3.3 Mo's algorithm

4 Strings

4.1 Z-function

4.2 Rolling hash


```cpp
class Hash {
  #define op operator
private:
  inline static constexpr int SZ = 2, MOD[6] = {
    (int) 1e9+7, (int) 1e9+20041203, (int) 1e9+2277,
    (int) 1e9+5277, (int) 1e9+8277, (int) 1e9+9277,
  };
  int val[SZ] = {};
  static ll pw(ll a, ll n, ll mod) {
    ll answer = 1;
    for (; n > 0; a = a * a % mod, n >>= 1)
      if (n & 1) answer = answer * a % mod;
    return answer;
  }
public:
  Hash(ll num){EFOR(int,i,0,SZ)val[i]=num%MOD[i];}
  Hash(): Hash(0) {}
  int& op[] (int idx) {return val[idx];}
  bool op == (const Hash &oth) const {
    EFOR(int,i,0,SZ)if(oth.val[i]!=val[i])return false;
    return true;
  }
  bool op < (const Hash &oth) const {
    EFOR(int,i,0,SZ)
      if(oth.val[i]!=val[i]) return val[i]<oth.val[i];
    return false;
  }
  bool op <= (const Hash &oth) const {
    EFOR(int, i, 0, SZ)
      if(oth.val[i]!=val[i])return val[i]<oth.val[i];
    return true;
  }
  bool op != (const Hash&oth)const{return not op==(oth);}
  bool op > (const Hash&oth)const{return not op<=(oth);}
  bool op >= (const Hash&oth)const{return not op<(oth);}
  Hash& op += (Hash &oth) {
    EFOR(int,i,0,SZ)val[i]=(val[i]+oth[i])%MOD[i];
    return *this;
  }
  Hash& op -= (Hash &oth) {
    EFOR(int,i,0,SZ)val[i]=(val[i]+MOD[i]-oth[i])%MOD[i];
    return *this;
  }
  Hash& op *= (Hash &oth) {
    EFOR(int,i,0,SZ)val[i]=(1ll*val[i]*oth[i])%MOD[i];
    return *this;
  }
  Hash inv() const {
    Hash ans; EFOR(int, i, 0, SZ)
      ans[i] = pw(val[i], MOD[i]-2, MOD[i]);
    return ans;
  }
  Hash& op/=(const Hash&oth){return op*=(oth.inv());}
  Hash& op+=(Hash&&oth){ return op+=(oth);}
  Hash& op-=(Hash&&oth){ return op-=(oth);}
  Hash& op*=(Hash&&oth){ return op*=(oth);}
  Hash& op/=(Hash&&oth){ return op/=(oth);}
  friend Hash op+(Hash a,Hash b){return(a += b);}
  friend Hash op-(Hash a,Hash b){return(a -= b);}
  friend Hash op*(Hash a,Hash b){return(a *= b);}
  friend Hash op/(Hash a,Hash b){return(a /= b);}
  friend ostream& op << (ostream& out, const Hash &num) {
    out << "(";
    EFOR(int,i,0,SZ)out<<num.val[i]<<",)"[i==SZ-1];
    return out;
  }
};

class HashString {
private:
  inline static constexpr int MAX = 1e6, BASE = 311;
  inline static Hash _p[MAX + 1], _i[MAX + 1];
  Hash *sF, *sB; int len;
  inline static bool _init = false;
public:
  static void init() {
    _init = true; _p[0] = 1;
    FOR(int, i, 1, MAX) _p[i] = _p[i-1] * BASE;
    _i[MAX] = _p[MAX].inv();
    REV(int, i, MAX-1, 0) _i[i] = _i[i + 1] * BASE;
    FOR(int, i, 0, MAX) assert(_i[i] * _p[i] == 1);
  }
  HashString(string s = "") {
    if (not _init) init(); len = s.size();
    sF=new Hash[len+1],sB=new Hash[len+2];
    sF[0]=sB[len+1]=0;
    FOR(int,i,1,len)sF[i]=sF[i-1]+s[i-1]*_p[i];
    REV(int,i,len,1)sB[i]=sB[i+1]+_p[len-i+1]*s[i-1];
  }
  ~HashString() {}

  int getLength() const {return len;}
  Hash getF(int l,int r){return(sF[r]-sF[l-1])*_i[l-1];}
  Hash getB(int l,int r){return(sB[l]-sB[r+1])*_i[len-r];}
  Hash getF(int idx) {return sF[idx];}
  Hash getB(int idx) {return sB[idx];}
};
```

4.3 Suffix array + LCP

4.4 Aho–Corasick

Per-pattern counts: `hit[u]++` at each step of the scan, then for u in reverse `order`: `hit[link[u]] += hit[u]`; pattern i occurs `hit[end`$_i$`]` times.
Online inserts: log groups of sizes 2^k; insert = merge the smaller groups and rebuild; a query sums over all groups.

```cpp
// patterns over the K characters first .. first+K-1; node 0 = root
template<int K = 26, char first = 'a'> struct AhoCorasick {
  struct Node { int nxt[K], link = 0; ll cnt = 0; Node() { fill(nxt, nxt + K, -1); } };
  vector<Node> t = vector<Node>(1);
  vector<int> order;                          // BFS order after build()

  int add(const string &s) {                  // returns the node where s ends
    int u = 0;
    for (char ch: s) {
      int c = ch - first;
      if (t[u].nxt[c] < 0) { t[u].nxt[c] = t.size(); t.emplace_back(); }
      u = t[u].nxt[c];
    }
    t[u].cnt++;
    return u;
  }

  void build() {                              // failure links + goto table; cnt[u] = #patterns ending at u's suffixes
    queue<int> q; q.push(0);
    while (!q.empty()) {
      int u = q.front(); q.pop(); order.push_back(u);
      for (int c = 0; c < K; c++) {
        int &v = t[u].nxt[c], w = u ? t[t[u].link].nxt[c] : 0;
        if (v < 0) v = w;
        else { t[v].link = w; t[v].cnt += t[w].cnt; q.push(v); }
      }
    }
  }

  ll countMatch(const string &s) {            // total occurrences of all patterns in s
    ll r = 0; int u = 0;
```

```
    for (char ch: s) { u = t[u].nxt[ch - first]; r +=
    ↪  t[u].cnt; }
    return r;
  }
};
```

4.5 Suffix automaton

4.6 Eertree

4.7 Lyndon factorisation (Duval)

5 Geometry

5.1 Point / line primitives

5.2 Polygon

5.3 Convex hull

5.4 Circles

5.5 Half-plane intersection

5.6 Closest pair / kd-tree

5.7 3D

6 Math

6.1 Number theory

6.1.1 Modint

```
typedef unsigned long long ull;
struct modint{
  inline static /*const*/ ll MOD = 998244353;
  int v;
  static inline ll mod(ll num) {
    ll val=num-ull((__uint128_t(-1ULL/MOD)*num)>>64)*MOD;
    return val-(val>=MOD)*MOD;
  }
  modint inv() const{
    ll answer = 1, a = v, n = MOD - 2;
    while (n) {
      if (n & 1) answer = mod(answer * a);
      a = mod(a * a); n >>= 1;
    }
    return answer;
  }

  modint(ll a = 0)
    { v=(a<0)?(MOD-mod(-a)):mod(a);v-=(v>=MOD)*MOD; }
  inline modint& operator += (modint b)
    { v+=b.v; v-=(v>=MOD)*MOD; return *this; }
  inline modint& operator -= (modint b)
    { v+=MOD-b.v; v-=(v>=MOD)*MOD; return*this; }
  inline modint& operator *= (modint b)
    { v = mod(1ll * v * b.v); return *this; }
  inline modint& operator /= (modint b)
    { return (*this)*=b.inv(); }
  inline modint& operator ^= (ll n) {
    modint a = v; v = 1;
    while (n) {if (n & 1) *this *= a; a *= a, n >>= 1;}
    return *this;
  }
};
#define NEWOP(op, op2) inline modint operator op\
(modint a, modint b) {return a op2 b;}
NEWOP(+,+=);NEWOP(-,-=);NEWOP(*,*=);NEWOP(/,/=);
#undef NEWOP
#define NEWCMP(op) inline bool operator op\
(modint a, modint b) {return a.v op b.v;}
NEWCMP(==);NEWCMP(!=);NEWCMP(<);NEWCMP(>);NEWCMP(<=);NEWCMP(>=);
#undef NEWCMP
inline modint operator - (modint a){return -a.v;}
inline modint operator^(modint a, ll n){return a^=n;}
inline istream& operator >> (istream& s, modint &i)
  { ll tmp; s >> tmp; i = tmp; return s; }
inline ostream& operator << (ostream& s, modint i)
  {return s << i.v;}
```

6.1.2 Bigint

```
#include "../contest/template.hpp"
#include "ntt.hpp"

// potential MLE in reserve() && resize()
class bigint {
using vc = vector<char>;
private:
  bool neg = false;
  vc ds;
  inline bigint&flipNega(){neg=!neg;return *this;}
  inline void reformat() {
    while ((int)ds.size() > 1 && *(ds.rbegin()) == 0)
    ↪  ds.pop_back();
    if (ds.size() == 1 && ds[0] == 0) neg = false; }
  inline bigint& _pD(const bigint&num, int p10=0) {
    resize(max(size(), num.size() + p10)); bool nho = 0;
    EFOR(int, i, p10, num.size() + p10) {
      ds[i] += nho + num.ds[i-p10];
      nho = ds[i] > 9; ds[i] -= 10 * nho; }
    EFOR(int, i, num.size() + p10, size()) {
      ds[i]+=nho; nho=ds[i]>9; ds[i]-=10*nho;
      if (nho == 0) break; }
    if (nho) push_back(nho); return *this; }
  inline bool ltAbs(const bigint&oth,bool xV,bool
  ↪  eqV)const{
    if (size()!=oth.size())return size()<oth.size();
    REV(int, i, size()-1, 0) if (ds[i] != oth.ds[i])
      return (ds[i] < oth.ds[i]) xor xV;
    return eqV; }
  inline bigint& _mD(const bigint &num) {
    bool flip = ltAbs(num, false, false);
    const vc *big = &(flip ? num.ds : ds),
         *small = &(flip ? ds : num.ds);
    resize(big->size());

    bool nho = 0;
    EFOR(int, i, 0, small->size()) {
      ds[i] = (*big)[i] - (nho + (*small)[i]);
      nho = ds[i] < 0; ds[i] += 10 * nho; }
    EFOR(int, i, small->size(), size()) {
      ds[i] = (*big)[i] - nho; nho = ds[i] < 0;
      ds[i] += 10 * nho; if (nho == 0) break; }
    if (nho or flip) flipNega();
    if (nho) push_back(nho); else reformat();
    return *this; }

public:
  inline static string _I;
  /* constructors */
  bigint() { ds = {0}; }
  bigint(ll num) {
    if(num == 0){ds={0};return;} ds.reserve(18);
    while (num) ds.push_back(num%10), num/=10; }
  bigint(string s) {
    ds.reserve(s.size());
    REV(int, j, (int)s.size()-1, 0)
      if (s[j] != '-') ds.push_back(s[j] - '0');
      else neg = true;
    reformat(); }
  bigint(vc &ds, bool neg):ds(ds),neg(neg){reformat();}
  bigint(vc &&ds, bool neg):ds(ds),neg(neg){reformat();}
  /* access data */
  inline bool isN() const { return neg; }
  inline char&operator[](int idx) { return ds[idx]; }
  /* copy vector api */
  inline int size() const { return ds.size(); }
  inline bool empty() const { return ds.empty(); }
  inline void pop_back(){ ds.pop_back(); }
  inline void reserve(int sz) { ds.reserve(sz); }
  inline void resize(int sz) { ds.resize(sz); }
  inline void push_back(char c){ ds.push_back(c); }
  /* comparator */
  inline bool operator == (const bigint& oth) const {
    if(isN()!=oth.isN()||size()!=(int)oth.size())return 0;
    REV(int,i,size()-1,0)if(ds[i]!=oth.ds[i])return false;
    return true; }
  inline bool operator < (const bigint& oth) const
    { return(isN()!=oth.isN())?isN():ltAbs(oth,isN(),0); }
  inline bool operator <= (const bigint& oth) const
    { return(isN()!=oth.isN())?isN():ltAbs(oth,isN(),1); }
  inline bool operator != (const bigint &num) const
    { return not operator==(num); }
  inline bool operator >= (const bigint &num) const
    { return not operator<(num); }
  inline bool operator > (const bigint &num) const
    { return not operator<=(num); }
```

```
  // fix rlvalue error/improve performance by:
  // inline bool operator ...  (bigint &&num) const
  //   { return operator ... (num); }

  /* operator + - * / */
  inline bigint operator - () const
    { return bigint(*this).flipNega(); }
  inline bigint& operator += (const bigint& num)
    {return (num.isN()==isN())?_pD(num):_mD(num);}
  inline bigint& operator -= (const bigint& num)
    {return (num.isN() == isN())?_mD(num):_pD(num);}
  inline bigint operator + (const bigint& num)
    { bigint res(*this); res += num; return res; }
  inline bigint operator - (const bigint& num)
    { bigint res(*this); res -= num; return res; }
  inline bigint operator * (const bigint &num) const {
    if (*this==0||num==0||empty()||num.empty())return 0;
    if (size() == 1 or num.size() == 1) {
      const vector<char> &D =(size()==1)?num.ds:ds;
      char mul = (size() == 1) ? ds[0] : num.ds[0];
      bigint ans; ans.resize(D.size()); char nho = 0;
      EFOR(int, i, 0, D.size()) {
        ans[i] = D[i] * mul + nho; nho = ans[i] / 10;
        ans[i] -= 10 * nho;
      }
      if (nho) ans.push_back(nho);
      if (isN() xor num.isN()) ans.flipNega();
      return ans;
    }
    /* FFT */
    FFT::buildRoot();
    vector<int> a(ds.begin(), ds.end());
    vector<int> b(num.ds.begin(), num.ds.end());
    vector<modint> newAns = FFT::multiply(a, b);
    bigint ans; ans.resize(newAns.size());
    int nho = 0;
    EFOR(int, i, 0, ans.size()){
      int tmp = newAns[i].v + nho;
      nho = tmp / 10;
      ans[i] = tmp - nho * 10;
    }
    while (nho>0) {ans.push_back(nho%10); nho/=10;}
    /* Karatsuba */
    // int mxSz = max(size(), num.size()), B = mxSz / 2;
    // bigint a0(vector<char>(ds.begin(), ds.begin() +
    ↪  min(B, size())), false),
    // a1(vector<char>(ds.begin()+min(B, size()), ds.end()
    ↪  ), false),
    // b0(vector<char>(num.ds.begin(),
    ↪  num.ds.begin()+min(B, num.size())), false),
    // b1(vector<char>(num.ds.begin()+min(B, num.size()),
    ↪  num.ds.end() ), false);
    //ans=a0*b0;bigint
    ↪  z2=a1*b1,z1=(a0+a1)*(b0+b1)-(z2+ans);
    // ans._pD(z2, B*2)._pD(z1, B);
    if (isN() xor num.isN()) ans.flipNega();
    ans.reformat(); return ans;
  }
  friend istream& operator>>(istream&inp, bigint&num)
    {inp>>bigint::_I;num=bigint(bigint::_I);return inp;}
  friend ostream&operator<<(ostream&out,const bigint&num){
    if (num.isN()) out << '-';
    REV(int,i,(int)num.size()-1,0)out<<char(num
    ↪  .ds[i]+'0');
    return out;
  }
};
```

6.1.3 Ext-gcd, CRT

6.1.4 Sieve (factorable)

```
// remember to call sieve() before any factorize()
namespace Eratos_Factorable {
  constexpr ll MAX = 100000000;
  constexpr ll EST = 53000000; // 1.1*MAX/ln(MAX)
  int smallDiv[MAX + 1] = {};
  int primes[EST], cntPrime = 0;
  inline void sieve(int size = MAX) {
    memset(smallDiv, false, sizeof(smallDiv));
    primes[++cntPrime] = 2; primes[++cntPrime] = 3;
    for (int i=2; i<=size; i += 2) smallDiv[i] = 2;
    for (int i=3; i<=size; i += 6) smallDiv[i] = 3;
    for (int mul = 1; 6 * mul - 1 <= size; mul++) {
      bool pass = false;
      for(int i:{6*mul-1,6*mul+1})if(!smallDiv[i]){
        primes[++cntPrime] = i; smallDiv[i] = i;
        for (ll j = 1ll*i*i; j <= size; j += i*2)
          if (not smallDiv[j]) smallDiv[j] = i;
        if (1ll * i * i > size) pass = true;
      }
      if (pass) break;
    }
  }
  inline vector<int> factorize(ll number) {
    vector<int> ans;
    while (number > 1) {
      int d = smallDiv[number];
      while (number % d == 0)
        ans.push_back(d), number /= d;
    }
    return ans;
  }
}
```

6.1.5 Miller–Rabin

```
#define MAX 1000001
#define MOD 1000000007

namespace MillerRabin{
  typedef __int128_t i128;
  constexpr int SMALL_PRIMES[12] =
  ↪  {2,3,5,7,11,13,17,19,23,29,31,37};

  inline ll _power(ll a, ll n, ll mod) {
    ll ans = 1;
    while (n) {
      if (n & 1) ans = (i128)ans * a % mod;
      a = (i128)a * a % mod; n >>= 1;
    }
    return ans;
  }
  inline bool _fermatCheck(ll n, ll a, ll pw, int p2) {
    ll num = _power(a, pw, n);
    if (num == 1 || num == n - 1) return true;
    FOR(int, i, 1, p2-1) {
      num = (i128) num * num % n;
      if (num == n - 1) return true;
    }
    return false;
  }

  inline bool checkPrime(const ll n) {
    if (n == 2 || n == 3 || n == 5 || n == 7) return true;
    if (n < 11 || (n & 1) == 0) return false;
    ll d = n-1; int p2 = 0;
    while ((d & 1) == 0) d >>= 1, p2++;
    for (int a: SMALL_PRIMES)
      if (n == a) return true;
      else if (not _fermatCheck(n, a, d, p2))
        return false;
    return true;
  }
}
```

6.1.6 Pollard rho

```
#include "millerrabin.hpp"
randomize;
namespace Pollards{
  typedef __int128_t i128;
  #define sqp1(a, b, mod) (((i128(a)*(a)+(b)))%(mod))
  #define rand (mt()%1'000'000'000'000 + 1)
  inline vector<ll> rho(ll n) {
    if (n == 1) return {};
    if (MillerRabin::checkPrime(n)) return {n};
    vector<ll> ans;
    if (n % 2 == 0) {
      while (n % 2 == 0) ans.push_back(2), n /= 2;
      vector<ll> others = rho(n);
      ans.insert(ans.end(), others.begin(), others.end());
      return ans;
    }
    ll x = 2, y = 2, g = 1, b = 1;
    while (g == 1) {
      x=sqp1(x,b,n); y=sqp1(y,b,n); y=sqp1(y,b,n);
      g = __gcd(abs(x-y), n);
      if (g == n) x=y=rand, b=rand, g=1;
    }
    vector<ll> tmp1 = rho(g), tmp2 = rho(n / g);
    ans.insert(ans.end(), tmp1.begin(), tmp1.end());
    ans.insert(ans.end(), tmp2.begin(), tmp2.end());
    return ans;
```

```
  }
}
```

6.1.7 Discrete log / discrete sqrt

6.1.8 Floor sum

6.2 Linear algebra

6.2.1 Gauss–Jordan

```cpp
#include "../contest/template.hpp"
#include "modint.hpp"
#define MAX 500
namespace Gaussian {
struct Equation {
  vector<modint> cf; modint sum; // cf coef
  void div(modint d){ for(modint&num:cf)num/=d;sum/=d; }
  void minus(Equation &eq, modint mul) {
    FOR(int,i,0,(int)cf.size()-1)cf[i]-=eq.cf[i]*mul;
    sum -= eq.sum * mul; } };
struct Elimination{
  int SZ=0;Elimination(){}Elimination(int SZ):SZ(SZ){}
  Equation eqs[MAX]; int cnt = 0; int col[MAX];

  void add(Equation eq) { eqs[cnt++] = eq; }
  int size() {return SZ;}    int count() {return cnt;}
  pair<vector<modint>, vector<vector<modint>>> solve() {
    int ln = 0;
    FOR(int, i, 0, size()-1) {
      int ptr = ln;
      while (ptr < count() and eqs[ptr].cf[i] == 0) ptr++;
      if (ptr==count())continue; swap(eqs[ln],eqs[ptr]);
      eqs[ln].div(eqs[ln].cf[i]);
      FOR(int, j, 0, count()-1) if (j != ln)
        eqs[j].minus(eqs[ln], eqs[j].cf[i]);
      col[ln++] = i; }
    FOR(int,i,ln,count()-1) if(eqs[i].sum!=0) return{};
    vector<modint> sol(SZ); vector<vector<modint>> basis;
    REV(int, i, size()-1, 0) {
      if (i != col[ln-1]) {
        vector<modint> cur(SZ); cur[i] = 1;
        FOR(int,row,0,ln-1)cur[col[row]]=-eqs[row].cf[i];
        basis.push_back(cur); continue; }
      sol[i] = eqs[--ln].sum; }
    return {sol, basis}; } }; }
```

6.2.2 Simplex

6.3 Polynomials & transforms

6.3.1 FFT

```cpp
#include "../contest/template.hpp"

#define MAX 1000001
#define MOD 1000000007

typedef complex<ld> cd;
namespace FFT{
  const ld TAU = acos(-1) * 2;
    // 3.14159'26535'89793'23846'26433'83279'50288'41971
    // '69399'37510'58 * 2;

  constexpr int BIT = 20, MAX_LEN = 1 << BIT;
  vector<int> _rev[BIT + 1];
  cd _root[MAX_LEN + 1];
  void buildRoot() {
    _root[0] = _root[MAX_LEN] = 1;
    for (int i=BIT-1, dist=1<<(BIT-1); i>=0; i--,
    ↪  dist>>=1) {
      cd w = polar(ld(1.0), TAU * dist / MAX_LEN);
      for (int pos = 0; pos < MAX_LEN; pos += 2 * dist)
        _root[pos + dist] = _root[pos] * w;
    }
    _rev[0] = {0};
    for (int bit=1, len=2; len<=MAX_LEN; bit++, len*=2) {
      _rev[bit].resize(len, 0);
      for (int i = 0; i < len; i++)
        _rev[bit][i]=((i&1)<<(bit-1))|_rev[bit-1][i/2];
    }
  }
  void dft(vector<cd> &poly, bool invert = false) {
    assert((cntBit((int)poly.size()) == 1));
    const int n = poly.size(), coef = MAX_LEN / n;
    for (int dist=n/2, span=n; dist>0; dist/=2, span/=2)
    for (int pos = 0; pos < dist; pos++)
    for (int i=pos, k=0; i<n; i+=span, k+=dist) {
      int len = log2(n / span);
      int newK = _rev[len][k / dist] * dist;
      int tmp = (invert ? (n - newK) : newK) * coef;
      cd a = poly[i], b = _root[tmp] * poly[i + dist];
      poly[i] = a + b, poly[i + dist] = a - b;
    }
    if (invert) for (cd &x: poly) x /= n;
  }
  template<class T>vector<T>conv(vector<T>_a,vector<T>_b){
    int len = int(_a.size() + _b.size()) - 1;
    int bit = log2(len)+(cntBit(len)>1); len=1<<bit;
    vector<cd> a(len);
    for (int i = 0; i < len; i++)
      a[i] = cd((i < _a.size())?_a[i]:0,
      ↪  (i<_b.size())?_b[i]:0);
    dft(a, false);
    for (int i = 0; i < len; i++)
      if (i < _rev[bit][i]) swap(a[i], a[_rev[bit][i]]);
    for (int i = 0; i < len; i++) a[i] *= a[i];
    vector<cd> b(a.begin(), a.end());
    for (int i = 0; i < len; i++)
      b[i] = a[i] - conj(a[-i & (len-1)]); //(n-i)%n
    dft(b, true);
    for (int i = 0; i < len; i++)
      if (i < _rev[bit][i]) swap(b[i], b[_rev[bit][i]]);

    while (len > 0 && b[len - 1].imag() < 1e-9) len--;
    vector<T> ans(len);
    FOR(int, i, 0, len-1) ans[i]=round(b[i].imag()/4.0);
    return ans;
  }
}
```

6.3.2 NTT

k-th root of unity, mod p: ω_k satisfies $\begin{cases}\omega_k^k \equiv 1 \\ \omega_k^i \not\equiv \omega_k^j \quad (i \not\equiv j)\end{cases}$
$p = c \times 2^k + 1 \Rightarrow w_{2^k} = g^c$ w/ any $g : \gcd(g, p) = 1$
only works for p with big k (e.g. 998244353 w/ $k = 23$ below)

```cpp
#include "modint.hpp"

// MOD=998244353=c*2^k+1 => ROOT=g^c, any g: gcd(g,MOD)=1
namespace FFT{
  constexpr int K_MOD = 23, BIT = 22, MAX_LEN = 1 << BIT;
  constexpr int ROOT = 15311432;

  modint _root[MAX_LEN + 1];
  vector<int> _rev[BIT + 1];
  bool _built = false;
  void buildRoot() {
    if (_built) return; _built = true;
    _root[0] = 1; modint mul=ROOT;
    FOR(int, i, 1, K_MOD-BIT) mul*=mul;
    FOR(int, i, 1, MAX_LEN) _root[i] = _root[i-1] * mul;
    for (int bit=0, len=1; len<=MAX_LEN; bit++, len*=2) {
      _rev[bit].resize((1 << bit), 0);
      for (int i = 1; i < (1 << bit); i++)
        _rev[bit][i]=((i&1)<<(bit-1))|_rev[bit-1][i/2];
    }
  }

  void transform(vector<modint> &v, bool invert){
    const int len = v.size(), coef = MAX_LEN / len;
    const int bit = log2(len);
    FOR(int, i, 0, len-1) if (i < _rev[bit][i])
      swap(v[i], v[_rev[bit][i]]);

    for (int jmp=1, span=2; span<=len; jmp*=2, span*=2)
    for (int beg = 0; beg < len; beg += span)
    for (int i = 0; i < jmp; i++) {
      int k=coef*len/jmp*i/2; if (invert) k=MAX_LEN-k;
      modint a = v[beg+i], b = _root[k] * v[beg+i+jmp];
      v[beg + i] = a + b; v[beg + i + jmp] = a - b;
    }
    if (invert) FOR(int, i, 0, len-1) v[i] /= len;
  }
  template<class T>
  vector<modint> multiply(vector<T> &_a, vector<T> &_b) {
    int len = int(_a.size() + _b.size()) - 1;
    len = 1 << (log2(len) + (cntBit(len) > 1));
    vector<modint>a(_a.begin(),_a.end()); a.resize(len,0);
    vector<modint>b(_b.begin(),_b.end()); b.resize(len,0);
```

```
44      transform(a, false); transform(b, false);
45      FOR(int, i, 0, len - 1) a[i] *= b[i];
46      transform(a, true); return a;
47    }
48  }
```


6.3.3 FWHT

6.3.4 FPS: inv, log, exp, div

6.3.5 Berlekamp–Massey + Bostan–Mori

6.3.6 Lagrange interpolation

6.4 Numerical

6.4.1 Simpson integration

7 Interactive & Multi-pass

7.1 Checklists

7.1.1 Interactive

7.1.2 Multi-pass

7.1.3 Noise taxonomy

7.2 Codes

7.2.1 Linear codes: Reed–Solomon, GF(2), stuck bits

7.2.2 Bit-level codes: self-delimiting, VT, pairs, checksum

7.2.3 Hamming SECDED

7.3 Encoding

7.3.1 Rank / unrank

7.3.2 Tree canonical form (AHU)

8 Misc

8.1 DP optimisation: D&C, Knuth, Aliens

8.2 Ternary search + simulated annealing

9 Math Reference

9.1 Counting

9.2 Number theory

9.3 Sums & generating functions

9.4 Games

9.5 Probability & identities